

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: COMPUTER NETWORKS
COURSE CODE: CSA07

COURSE OUTCOME COVERED

CO5: Measure network performance using key metrics with simulation tools. (BL4,BL5)

Assessment Tool Weightage: 30%

Annexure A

Pseudocode Writing Task (Algorithm Design)

1. In a network simulation using Cisco Packet Tracer, a student records total data received as 8000 bits in 4 seconds and observes that 200 packets were sent while 180 packets were received. Write a pseudocode algorithm to calculate the network throughput in bits per second and the packet loss percentage based on the given values. The algorithm should include validation to ensure that time and total packets sent are not zero to avoid errors. It should display both the throughput and packet loss results clearly. Add logic to classify the network as Efficient if throughput is greater than 1500 bps and packet loss is less than 10%, otherwise classify it as Inefficient. Finally, ensure the algorithm outputs a complete summary of the network performance based on the calculated metrics.
2. An organization has multiple network links connecting its branches. The administrator wants to monitor bandwidth utilization and identify overloaded links. Write a pseudocode algorithm that periodically collects the bandwidth usage of each network link and stores the values in an array or list. Use a loop to continuously monitor all links. Add conditions to detect when bandwidth utilization exceeds 80% and recommend switching traffic to a backup link. Explain how this algorithm supports efficient bandwidth management and prevents network congestion.
3. A company has six branch offices connected through multiple network links. Each link has different bandwidth, delay, and utilization values, which change throughout the day. The network administrator wants to continuously identify the best communication path between the headquarters and each branch based on overall link quality rather than just the shortest distance. Write a pseudocode algorithm that periodically collects the bandwidth, delay, and utilization of every network link and stores the values in suitable arrays or lists. Use a loop to repeat

the monitoring process at regular intervals. Calculate a link quality score for each path using the collected metrics and select the path with the highest score. Include conditions to detect when a link becomes overloaded or fails, automatically selecting an alternate path and generating an alert for the administrator. Explain how your algorithm improves network reliability, routing efficiency, and fault tolerance while adapting to changing network conditions.

RUBRICS FOR CALCULATION

Criteria	Weight ag e	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Question 1 – Network Throughput and Packet Loss

In a Cisco Packet Tracer network simulation, total data received is 8000 bits in 4 seconds. A total of 200 packets are sent and 180 packets are received. The algorithm must calculate throughput and packet loss, validate time and packets sent, classify the network using the specified thresholds, and display a complete performance summary.

1.1 Problem Understanding

- Inputs: totalDataReceived = 8000 bits, time = 4 seconds, packetsSent = 200, packetsReceived = 180.
- Outputs: throughput in bits per second, packet loss percentage, network classification, and complete performance summary.
- Constraints/validation: time must not be zero; packets sent must not be zero; packet values must be valid; packets received should not exceed packets sent.
- Classification rule: Efficient only when throughput > 1500 bps AND packet loss < 10%; otherwise the network is Inefficient.

1.2 Calculation Verification

- Throughput = Total Data Received / Time = $8000 / 4 = 2000$ bps.
- Packets Lost = Packets Sent – Packets Received = $200 - 180 = 20$ packets.
- Packet Loss Percentage = (Packets Lost / Packets Sent) $\times 100 = (20 / 200) \times 100 = 10\%$.
- Classification: throughput is greater than 1500 bps, but packet loss is not less than 10% because it is exactly 10%. Therefore, the network is classified as Inefficient.

1.3 Level-4 Pseudocode

BEGIN

SET totalDataReceived $\leftarrow$ 8000

SET time $\leftarrow$ 4

SET packetsSent $\leftarrow$ 200

SET packetsReceived $\leftarrow$ 180

IF time $\leq$ 0 THEN

 DISPLAY "Error: Time must be greater than zero."

 STOP

END IF

IF packetsSent $\leq$ 0 THEN

 DISPLAY "Error: Total packets sent must be greater than zero."

 STOP

END IF

IF totalDataReceived $<$ 0 OR packetsReceived $<$ 0 OR packetsReceived $>$ packetsSent THEN

 DISPLAY "Error: Invalid network measurement values."

 STOP

END IF

SET throughput $\leftarrow$ totalDataReceived / time

SET packetsLost $\leftarrow$ packetsSent - packetsReceived

SET packetLossPercentage $\leftarrow$ (packetsLost / packetsSent) * 100

DISPLAY "NETWORK PERFORMANCE SUMMARY"

DISPLAY "Throughput: ", throughput, " bps"

DISPLAY "Packets Sent: ", packetsSent

DISPLAY "Packets Received: ", packetsReceived

DISPLAY "Packets Lost: ", packetsLost

DISPLAY "Packet Loss: ", packetLossPercentage, "%"

IF throughput > 1500 AND packetLossPercentage < 10 THEN

 SET classification ← "Efficient"

ELSE

 SET classification ← "Inefficient"

END IF

DISPLAY "Network Classification: ", classification

IF classification = "Efficient" THEN

 DISPLAY "Summary: The network meets both throughput and packet-loss targets."

ELSE

 DISPLAY "Summary: The network does not meet all performance targets."

END IF

END

1.4 Expected Output

- Throughput: 2000 bps
- Packets Lost: 20
- Packet Loss: 10%
- Network Classification: Inefficient
- Summary: The network does not meet all performance targets.

1.5 Rubric Evidence

- Problem Understanding: all four inputs, required outputs, threshold conditions, and validation constraints are explicitly identified.
- Algorithm Design: the sequence validates data, calculates both metrics, applies the exact classification rule, and produces a complete summary.
- Control Structures: IF conditions prevent invalid calculations and implement classification logic.
- Pseudocode Structure: BEGIN/END, SET, IF/THEN/ELSE, DISPLAY, and STOP provide consistent and readable structure.
- Completeness: valid and invalid input cases are handled, and the boundary condition of exactly 10% packet loss is correctly treated as Inefficient.

Question 2 – Periodic Bandwidth Utilization Monitoring

An organization has multiple network links connecting its branches. The administrator needs to periodically collect bandwidth usage, store the values in an array or list, continuously monitor all links, detect utilization above 80%, and recommend switching traffic to a backup link.

2.1 Problem Understanding

- Inputs: number of links, link identifiers, current bandwidth utilization for each link, monitoring interval, and backup-link availability.
- Data structure: arrays/lists store link IDs and their corresponding utilization values.
- Outputs: utilization values, overload status, and a recommendation to switch traffic when utilization exceeds 80%.
- Constraints: monitoring must continue periodically; utilization should be within 0%–100%; a backup link should be checked before recommending traffic migration.

2.2 Level-4 Pseudocode

BEGIN

SET linkCount $\leftarrow$ number of network links

CREATE array linkID[1..linkCount]

CREATE array utilization[1..linkCount]

CREATE array backupAvailable[1..linkCount]

SET monitoringInterval $\leftarrow$ 60 seconds

IF linkCount $\leq$ 0 THEN

 DISPLAY "Error: At least one network link is required."

 STOP

END IF

WHILE TRUE DO

 DISPLAY "Starting bandwidth monitoring cycle..."

 FOR $i \leftarrow 1$ TO linkCount DO

 utilization[i] $\leftarrow$ COLLECT_BANDWIDTH_UTILIZATION(linkID[i])

 IF utilization[i] < 0 OR utilization[i] > 100 THEN

 DISPLAY "Invalid utilization value for ", linkID[i]

 CONTINUE

 END IF

 DISPLAY "Link ", linkID[i], " Utilization: ", utilization[i], "%"

 IF utilization[i] > 80 THEN

 DISPLAY "ALERT: Link ", linkID[i], " is overloaded."

```

    backupAvailable[i] ← CHECK_BACKUP_LINK(linkID[i])

    IF backupAvailable[i] = TRUE THEN
        DISPLAY "Recommendation: Switch traffic from ", linkID[i],
            " to its backup link."
        SWITCH_TRAFFIC_TO_BACKUP(linkID[i])
    ELSE
        DISPLAY "Warning: No backup link available for ", linkID[i]
        DISPLAY "Recommendation: Reduce non-critical traffic and
investigate congestion."
    END IF
ELSE
    DISPLAY "Link ", linkID[i], " is within acceptable utilization."
END IF
END FOR

DISPLAY "Monitoring cycle completed."
WAIT monitoringInterval
END WHILE
END

```

2.3 How the Algorithm Supports Bandwidth Management

- Continuous monitoring detects rising utilization before severe congestion occurs.
- The 80% threshold provides an early warning point so traffic can be moved before the link becomes critically overloaded.

- Storing measurements in arrays/lists makes link-by-link comparison and repeated monitoring systematic.
- Checking backup availability before switching prevents the algorithm from recommending an unavailable route.
- The periodic loop supports proactive network management rather than waiting for packet loss or service failure.

2.4 Rubric Evidence

- Problem Understanding: identifies link measurements, storage, monitoring interval, overload threshold, backup availability, and outputs.
- Algorithm Design: collects, validates, stores, evaluates, and responds to utilization data in a complete sequence.
- Control Structures: WHILE provides continuous monitoring; FOR scans every link; IF/ELSE handles normal, overloaded, invalid, and no-backup cases.
- Pseudocode Structure: modular operations such as COLLECT_BANDWIDTH_UTILIZATION, CHECK_BACKUP_LINK, and SWITCH_TRAFFIC_TO_BACKUP make responsibilities clear.
- Completeness: handles invalid utilization, overloaded links, available backup links, unavailable backup links, normal links, and repeated monitoring.

Question 3 – Dynamic Best-Path Selection

A company has six branch offices connected through multiple links. Bandwidth, delay, and utilization vary throughout the day. The administrator must continuously identify the best path using overall link quality rather than shortest distance, select alternate paths when a link fails or becomes overloaded, and alert the administrator.

3.1 Problem Understanding

- Inputs: network links/paths, bandwidth, delay, utilization, link status, monitoring interval, and branch destinations.

- Stored metrics: arrays/lists for bandwidth, delay, utilization, status, and calculated path quality.
- Outputs: highest-quality path for each branch, alternate path when required, and administrator alert for overload/failure.
- Constraints: path quality must consider multiple metrics; overloaded or failed links must not be selected; monitoring repeats at regular intervals.

3.2 Link Quality Model

To combine metrics with different units, the algorithm normalizes bandwidth, delay, and utilization before calculating a score. A higher score is better. The following weighted model emphasizes available bandwidth while penalizing delay and utilization:

$$\text{NormalizedBandwidth} = \text{Bandwidth} / \text{MaximumBandwidth}$$

$$\text{NormalizedDelay} = \text{MinimumDelay} / \text{Delay}$$

$$\text{NormalizedUtilization} = 1 - (\text{Utilization} / 100)$$

$$\begin{aligned} \text{QualityScore} = & (0.50 * \text{NormalizedBandwidth}) \\ & + (0.30 * \text{NormalizedDelay}) \\ & + (0.20 * \text{NormalizedUtilization}) \end{aligned}$$

The score is intentionally based on overall link quality rather than physical distance. The weights can be tuned by the network administrator according to service requirements.

3.3 Level-4 Pseudocode

```
BEGIN
    SET branchCount ← 6
    SET monitoringInterval ← 60 seconds
```

```

CREATE list Branch[1..branchCount]
CREATE list Link[1..numberOfLinks]
CREATE arrays Bandwidth[], Delay[], Utilization[], Status[], QualityScore[]

WHILE TRUE DO
    DISPLAY "Starting dynamic path monitoring..."

    MaximumBandwidth ← 0
    MinimumDelay ← INFINITY

    FOR each link L in Link DO
        Bandwidth[L] ← COLLECT_BANDWIDTH(L)
        Delay[L] ← COLLECT_DELAY(L)
        Utilization[L] ← COLLECT_UTILIZATION(L)
        Status[L] ← CHECK_LINK_STATUS(L)

        IF Status[L] = "UP" AND Bandwidth[L] > MaximumBandwidth THEN
            MaximumBandwidth ← Bandwidth[L]
        END IF

        IF Status[L] = "UP" AND Delay[L] < MinimumDelay THEN
            MinimumDelay ← Delay[L]
        END IF
    END FOR

    IF MaximumBandwidth <= 0 OR MinimumDelay = INFINITY THEN
        DISPLAY "CRITICAL ALERT: No valid active links available."
    END IF
END WHILE

```

```
    WAIT monitoringInterval
    CONTINUE
END IF
```

```
FOR each link L in Link DO
    IF Status[L] = "DOWN" THEN
        QualityScore[L] ← -1
    ELSE IF Utilization[L] > 80 THEN
        QualityScore[L] ← -1
        DISPLAY "ALERT: Link ", L, " is overloaded."
    ELSE
        NormalizedBandwidth ← Bandwidth[L] / MaximumBandwidth
        NormalizedDelay ← MinimumDelay / Delay[L]
        NormalizedUtilization ← 1 - (Utilization[L] / 100)

        QualityScore[L] ←
            (0.50 * NormalizedBandwidth) +
            (0.30 * NormalizedDelay) +
            (0.20 * NormalizedUtilization)
    END IF
END FOR
```

```
FOR each branch B in Branch DO
    CandidatePaths ← FIND_AVAILABLE_PATHS_TO(B, Link, Status,
Utilization)
```

```
    IF CandidatePaths is EMPTY THEN
        DISPLAY "CRITICAL ALERT: No available path to branch ", B
```

```

    CONTINUE
END IF

    BestPath ← path in CandidatePaths with HIGHEST TOTAL QUALITY
    SCORE

    IF CURRENT_PATH(B) is DOWN OR CURRENT_PATH(B) is
    OVERLOADED THEN
        AlternatePath ← BestPath

        IF AlternatePath exists THEN
            SWITCH_TRAFFIC(B, AlternatePath)
            DISPLAY "ALERT: Traffic for branch ", B,
                " switched to alternate path."
        ELSE
            DISPLAY "CRITICAL ALERT: No alternate path for branch ", B
        END IF
    ELSE
        IF BestPath is different from CURRENT_PATH(B) AND
            QUALITY_SCORE(BestPath) >
QUALITY_SCORE(CURRENT_PATH(B)) THEN
            SWITCH_TRAFFIC(B, BestPath)
            DISPLAY "Path optimized for branch ", B
        ELSE
            KEEP_CURRENT_PATH(B)
        END IF
    END IF
END FOR

```

```
    DISPLAY "Dynamic routing analysis completed."  
    WAIT monitoringInterval  
END WHILE  
END
```

3.4 How the Algorithm Improves Network Performance

- Reliability: failed links are excluded from candidate paths and traffic can be redirected to an available alternative.
- Routing efficiency: the selected path is based on bandwidth, delay, and utilization instead of shortest physical distance alone.
- Fault tolerance: multiple candidate paths allow traffic to continue when a primary link fails.
- Adaptability: metrics are recollected at every monitoring interval, so decisions reflect changing daytime network conditions.
- Congestion avoidance: links above 80% utilization are excluded from normal path selection and trigger an administrator alert.
- Operational awareness: alerts communicate failures, overloads, missing paths, and route changes to the administrator.

3.5 Rubric Evidence

- Problem Understanding: identifies all dynamic metrics, storage requirements, path-selection requirements, overload/failure conditions, alerts, and repeated monitoring.
- Algorithm Design: performs metric collection, normalization, scoring, candidate-path selection, failure handling, overload handling, and route optimization.

- Control Structures: WHILE repeatedly monitors; FOR processes links and branches; nested IF/ELSE conditions manage valid, failed, overloaded, and alternate-path cases.
- Pseudocode Structure: separates measurement, normalization, scoring, candidate-path discovery, selection, and traffic switching into logically distinct operations.
- Completeness: includes normal operation, failure, overload, no-path conditions, alternate-path switching, route improvement, and critical alerts.